03. React

Detailed beginner handbook - English and Arabic

Technologies: React, React DOM

What you will learn

Create a reusable component with changing state.

React builds an interface from components and updates the screen when state changes.

الهدف: بناء مثال صغير وفهمه خطوة بخطوة.

شرح بسيط: تعلم الفكرة أولا، ثم جرب المثال الصغير ببيانات اصطناعية فقط.

How to study

Study one lesson at a time. Type examples yourself, predict the result, run the code, then change one thing. Keep a notebook of new words, errors, root causes, and fixes.

Course map

Setup and mental model; ten core concepts; a guided mini-project; the real dashboard architecture; debugging; exercises and answers.

Lesson 1 - Setup and mental model

Prerequisites

Windows 10 or 11, VS Code, PowerShell, and permission to install learning dependencies. Use only synthetic data in tutorials and screenshots.

Setup sequence

1. Open PowerShell.
2. Go to learning-examples/03-react.
3. Read README.md.
4. Run npm install then npm run dev.
5. Read the complete error if it fails.
6. Change one safe value and rerun.

الخطوات: افتح مجلد المثال، شغل الأمر، غير قيمة واحدة، ولاحظ النتيجة.

Mental model

Treat React, React DOM as one layer in a system. Identify the input, the transformation or rule, the output, and possible failures. Understanding that flow is more valuable than memorising syntax.

Checkpoint

Explain: What problem does this technology solve? What is its input? What output should it produce? What can fail?

Lesson 2 - Components, JSX, rendering, and composition

What is it?

A component is a function that returns JSX. JSX becomes element descriptions. Composition places components inside one another so a screen is built from focused reusable parts.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In React, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
function Welcome() {
  return <h1>Hello, React!</h1>;
}
// Render <Welcome /> inside the root.
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Components, JSX, rendering, and composition.

هذا الدرس يشرح: composition. اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. Components, JSX, rendering, and

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 3 - Props, one-way data flow, and reusable component interfaces

What is it?

Props are read-only inputs supplied by a parent. Data flows downward; callbacks communicate actions upward. Small, explicit prop interfaces make components easier to understand and test.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In React, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
function Card({title, total}) {
  return <section><h2>{title}</h2><p>{total}</p></section>;
}
<Card title="January" total={12} />
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Props, one-way data flow, and reusable component interfaces.

اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. Props, one-way data flow, and reusable component interfaces هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 4 - State with useState and immutable updates

What is it?

useState stores data between renders. Call its setter instead of mutating existing objects or arrays; create a new value so React can detect and render the update.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In React, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
const [total, setTotal] = useState(12);
<button onClick={() => setTotal(total + 1)}>{total}</button>
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates State with useState and immutable updates.

هذا الدرس يشرح: State with useState and immutable updates. اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك.

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 5 - Events, forms, controlled inputs, and validation

What is it?

Controlled inputs receive value from state and update it in onChange. Submit handlers validate, prevent default navigation, report field errors, and invoke application actions.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In React, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
const [month,setMonth] = useState('Jan');
<form onSubmit={e => e.preventDefault()}>
 <input value={month} onChange={e => setMonth(e.target.value)} />
</form>
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Events, forms, controlled inputs, and validation.

هذا الدرس يشرح: Events, forms, controlled inputs, and validation. اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك.

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 6 - Effects with useEffect and cleanup

What is it?

useEffect synchronises with systems outside rendering, such as network requests or subscriptions. Dependencies control reruns; cleanup cancels or disconnects obsolete work.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In React, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
useEffect(() => {
  const controller = new AbortController();
  loadData(controller.signal);
  return () => controller.abort();
}, []);
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Effects with useEffect and cleanup.

يشرح: Effects with useEffect and cleanup. اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. هذا الدرس

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 7 - Lists, keys, conditional rendering, and empty states

What is it?

map renders lists; stable keys identify items across updates. Conditional rendering shows loading, empty, error, or content states instead of assuming data always exists.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In React, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
{loading ? <p>Loading...</p> : rows.length === 0 ? <p>No data</p> :
 rows.map(row => <p key={row.id}>{row.total}</p>)}
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Lists, keys, conditional rendering, and empty states.

هذا الدرس يشرح: Lists, keys, conditional rendering, and empty states. اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك.

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 8 - Derived values with useMemo and stable callbacks

What is it?

useMemo caches an expensive derived value and useCallback caches a function identity. Use them for measured problems, not automatically, because they add complexity.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In React, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
const grandTotal = useMemo(
  () => rows.reduce((sum,row) => sum + row.total, 0),
  [rows]
);
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Derived values with useMemo and stable callbacks.

هذا الدرس يشرح: Derived values with useMemo and stable callbacks. اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك.

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 9 - Sharing state, context, and custom hooks

What is it?

Lift shared state to the nearest common owner. Context shares broadly needed values. A custom hook extracts reusable stateful behaviour without creating visible UI.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In React, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
function useDocumentTitle(title) {
  useEffect(() => { document.title = title; }, [title]);
}
// Context is for values needed by many descendants.
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Sharing state, context, and custom hooks.

اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. Sharing state, context, and custom hooks هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 10 - Loading, error, success, and accessibility patterns

What is it?

Disable repeated submissions, announce errors, preserve keyboard focus, label controls, and make loading state clear. Error boundaries handle unexpected rendering failures.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In React, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
<button disabled={saving} aria-busy={saving}>
  {saving ? 'Saving...' : 'Save'}
</button>
{error && <p role="alert">{error}</p>}
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Loading, error, success, and accessibility patterns.

هذا الدرس يشرح: Loading, error, success, and accessibility patterns. اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك.

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 11 - Component testing and avoiding common React anti-patterns

What is it?

Test visible behaviour and user actions. Avoid copying props into state, changing state during render, missing effect dependencies, unstable list keys, and giant all-purpose components.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In React, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
test('increases total', async () => {
  render(<Counter />);
  await user.click(screen.getByRole('button'));
  expect(screen.getByText('13')).toBeVisible();
});
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Component testing and avoiding common React anti-patterns.

هذا الدرس يشرح: Component testing and avoiding common React anti-patterns. اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك.

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 7 - Guided mini-project

Project goal

Create a reusable component with changing state.

Starting example

```
import {useState} from 'react';
export function Counter(){
 const [n,setN]=useState(12);
 return <button onClick={()=>setN(n+1)}>{n}</button>;
}
```

Read the code

Find imported tools or page elements. Identify the synthetic input. Find the function, event, route, or transformation that changes it. Finally, locate where the result is displayed, returned, saved, or packaged.

Build sequence

1. Run the untouched example.
2. Record the result.
3. Rename one unclear value.
4. Add a synthetic record.
5. Validate empty input.
6. Validate incorrect input.
7. Add a helpful error.
8. Rerun every case.
9. Compare with exercise-solution.
10. Explain every line.

Definition of done

Normal, empty, and invalid cases behave deliberately; no private data is used; and you can explain every line without reading the guide.

Lesson 8 - The real dashboard

Architecture connection

The application combines React, React DOM with the rest of the stack. The frontend gathers filters and files; the local API validates requests; the data layer transforms workbook rows; charts present aggregates; export tools create files; and the desktop layer packages the application for Windows.

Trace the upload feature

The user selects a workbook. The frontend sends it to the local API. The backend validates the file and sheets. The data layer creates safe tables. The API returns metadata. React updates state. Plotly displays results. Identify exactly where this guide's technology participates.

Privacy and quality

Do not place narrative, contact, or identifying fields in tutorials, logs, screenshots, tests, or exports. Use generated identifiers and synthetic totals. Validate at each boundary and collect only fields required for the calculation.

Architecture exercise

Draw five boxes: UI, API, Data Processing, Export, Desktop Packaging. Add arrows and label the data. Circle the box related to this guide and add two validation checks.

Lesson 9 - Debugging

Reliable debugging loop

1. Reproduce with the smallest input.
2. Read the first meaningful error.
3. Find the exact file and line.
4. Inspect value and type.
5. Compare expected and actual results.
6. Change one thing.
7. Rerun the same case.
8. Add a regression test.

Common beginner mistakes

- Wrong folder or command.
- Missing dependency or inactive environment.
- Misspelled file, variable, field, route, or import.
- Assuming input is always present.
- Mixing setup, processing, and display in one block.
- Hiding an exception rather than understanding it.
- Using real sensitive data in a test.

أخطاء شائعة: تشغيل الأمر من مجلد خاطئ، نسيان تثبيت التبعيات، أو تغيير أشياء كثيرة بمرة واحدة.

Error notebook

Record: command, expected result, actual error, root cause, smallest fix, and test added. This turns errors into reusable knowledge.

Lesson 10 - Exercises and answers

Exercises

1. Explain the technology in three sentences.
2. Recreate the example without copying.
3. Add one normal synthetic record.
4. Handle empty input.
5. Handle invalid input.
6. Improve unclear names.
7. Add or describe one test.
8. Locate the technology in the dashboard.
9. Record one error and root cause.
10. Add one mini-project feature.

تمرين: أضف شهرا أو قيمة آمنة جديدة، ثم اشرح ما حدث بكلماتك.

Answer guide

A strong solution is observable and explainable: the documented command works; output changes for the new record; empty and invalid cases have deliberate results; names communicate purpose; a test states an expected result; and the architecture explanation identifies the correct boundary.

Next step	Next: 04. Vite